

Deep Learning for Text Analytics

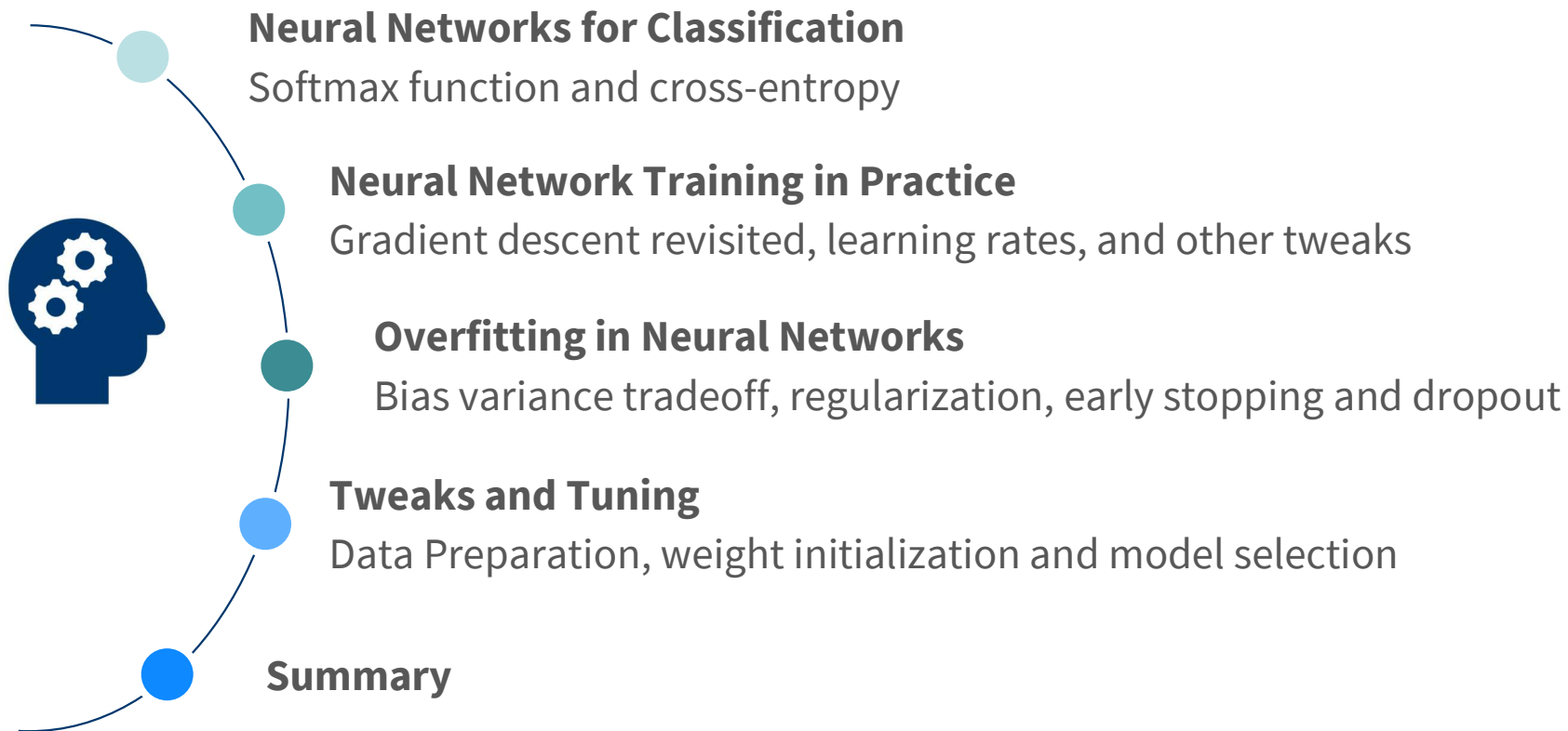


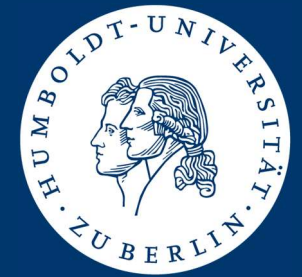
Foundations of Deep Learning (Part 2)

Stefan Lessmann [stefan.lessmann@hu-berlin.de]



Agenda





Neural Networks for Classification

Softmax function and cross-entropy

inker with a Neural Network Right Here in Your Browser.

Don't Worry, You Can't Break It. We Promise.

Epoch **000,000**

Learning rate **0.03**

Activation **Tanh**

Regularization **None**

Regularization rate **0**

Problem type **Classification**

DATA

Which dataset do you want to use?



Ratio of training to test data: 50%

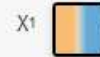
Noise: 0

Batch size: 10

REGENERATE

FEATURES

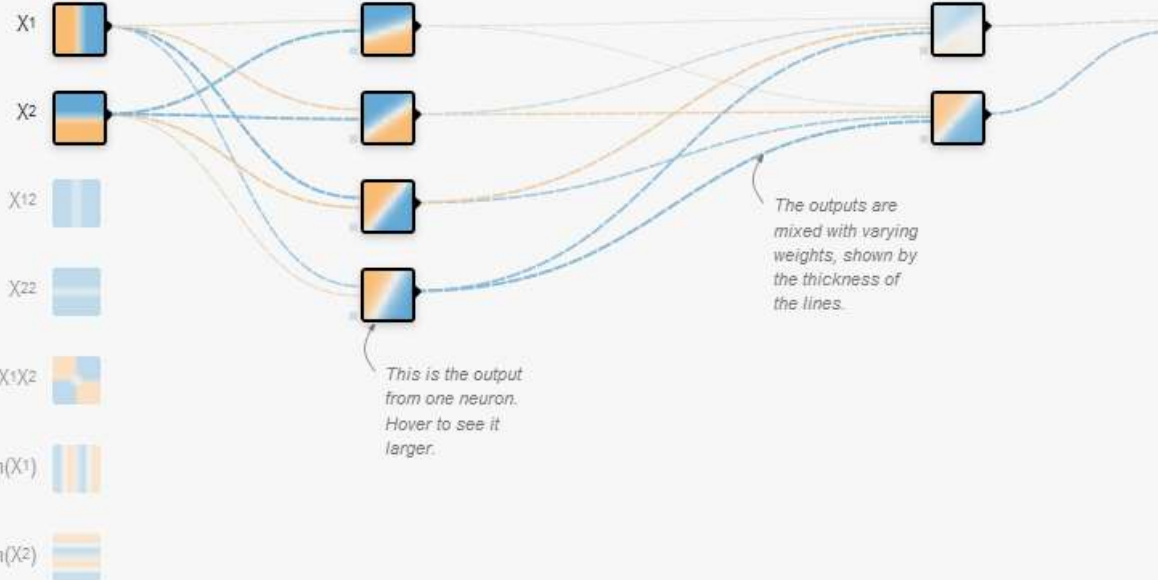
Which properties do you want to feed in?



+ - 2 HIDDEN LAYERS

+ -
4 neurons

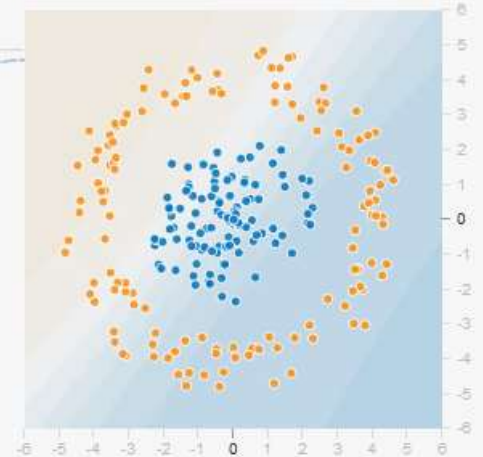
+ -
2 neurons



OUTPUT

Test loss 0.486

Training loss 0.530



Colors shows data, neuron and weight values.



Architecture of the Output Layer Determines NN Applications

A network architecture for binary classification

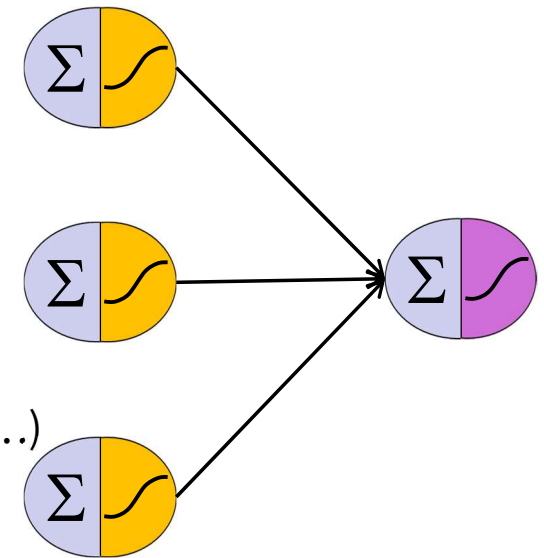
■ Output layer with one node

- Estimation of the entry probability of one of the two possible events (i.e., classes)
- Add sigmoid activation to output layer to guarantee that $\hat{y} \in [0,1]$

■ Train network using suitable function (see below)

■ Examples

- Customer scoring (e.g., loan approval)
 - Y : Did client pay back the loan?
 - X : Debt-to-income ratio, age, savings, employment type, ...
- Business failure prediction
 - Y : Did company file for insolvency?
 - X : Balance sheet and other accounting data
- Fraud detection (e.g., card transaction, tax statement, account login, ...)
 - Y : Is transaction legitimate?
 - X : Payment, purchased item, merchant,, ...



Architecture of the Output Layer Determines NN Applications

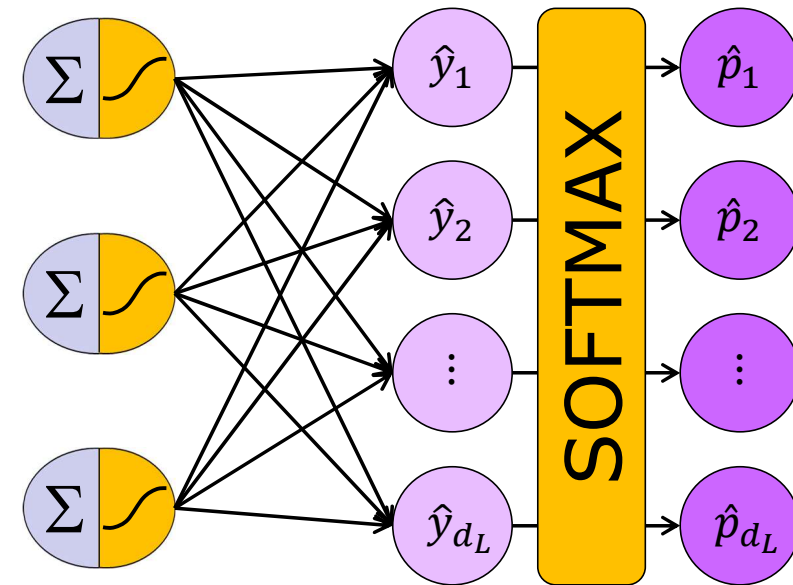
A network architecture for multi-class classification

■ Output layer with $d_L > 1$ nodes

- Estimate entry probability for each of $d_L > 1$ states
- Each output node models the entry probability for one of the classes/states
- Example: credit rating analysis

■ Softmax layer normalizes the NN output

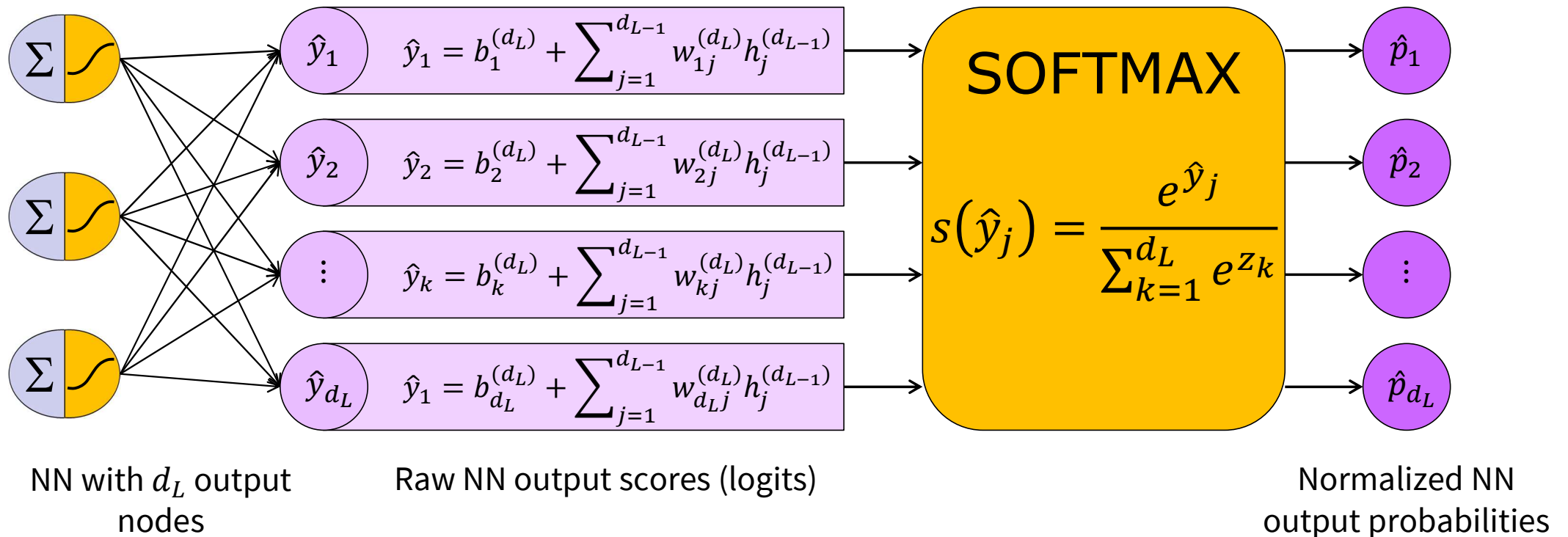
- Ensure $\sum_{o=1}^{d_L} \hat{p}_o = 1$
- Needed if classes are mutually exclusive
- Somewhat like an activation function applied to output layer
 - Just that an activation function processes the input of **one node**
 - Softmax processes all inputs to the **output layer**



The Softmax Function

Normalizing NN outputs for multi-class classification

$$\sum_{o=1}^{d_L} \hat{y}_o = 1$$



Cross-Entropy Loss Function for Multi-Class Classification Problems

- Multi-class problem with $d_L > 1$ mutually exclusive classes or states
- Let $c = 1, 2, \dots, d_L$ index individual classes
- Cross entropy loss

$$-\sum_{c=1}^{d_L} y_{ic} \log(f_w(\mathbf{X}_i)) \quad \text{where} \quad y_{ic} \in \{0,1\} \quad \text{and} \quad \sum_c y_{ic} = 1 \quad \forall i = 1, 2, \dots, n$$

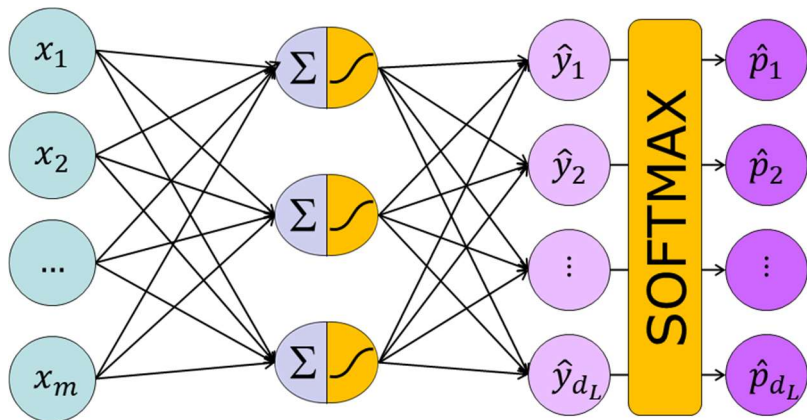
- Calculate separate loss for each class
- Sum over results (i.e., all classes)
- Obtain loss for a single data point

Cross-Entropy Loss Function for Multi-Class Classification Problems

One-hot encoding of class labels as (n, d_L) array to vectorize computations

$$\mathbf{X} = \{x_{ij}\}_{n \times m} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1m} \\ x_{21} & x_{22} & \cdots & x_{2m} \\ \vdots & \vdots & \cdots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nm} \end{pmatrix}$$

$$\mathbf{Y} = \{y_{ic}\}_{n \times d_L} = \begin{pmatrix} y_{11} & y_{12} & \cdots & y_{1d_L} \\ y_{21} & y_{22} & \cdots & y_{2d_L} \\ \vdots & \vdots & \cdots & \vdots \\ y_{n1} & y_{n2} & \cdots & y_{nd_L} \end{pmatrix}$$



$$\hat{\mathbf{Y}} = \{\hat{p}_{ic}\}_{n \times d_L} = \begin{pmatrix} \hat{p}_{11} & \hat{p}_{12} & \cdots & \hat{p}_{1d_L} \\ \hat{p}_{21} & \hat{p}_{22} & \cdots & \hat{p}_{2d_L} \\ \vdots & \vdots & \cdots & \vdots \\ \hat{p}_{n1} & \hat{p}_{n2} & \cdots & \hat{p}_{nd_L} \end{pmatrix}$$

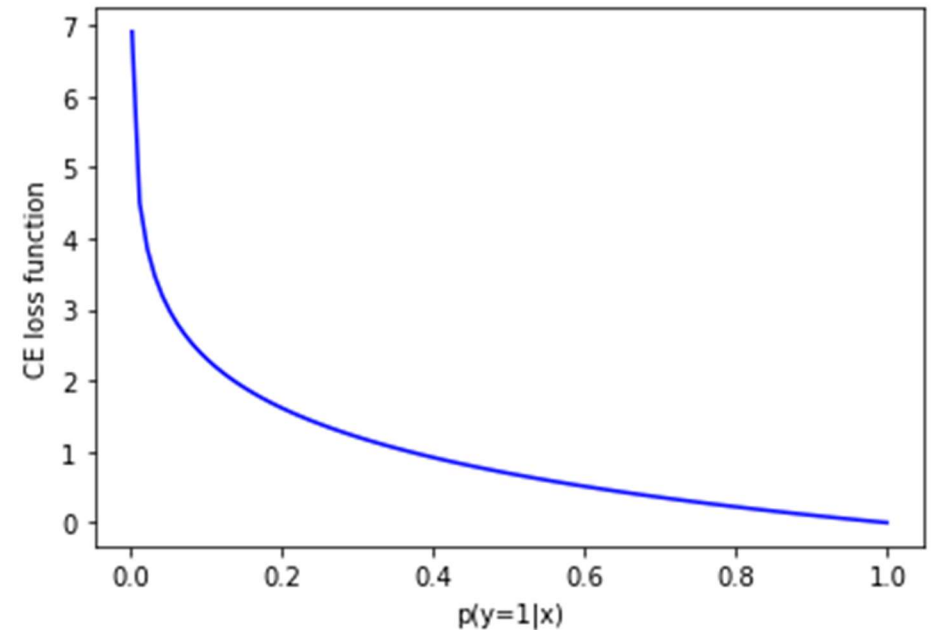
$$J(\mathbf{W}) = - \sum_{i=1}^n \sum_{c=1}^{d_L} y_{ic} \log(f_{\mathbf{W}}(\mathbf{X}_i)) = - \sum_{i=1}^n y_{i1} \log(\hat{p}_{11}) + y_{i2} \log(\hat{p}_{12}) + \cdots + y_{id_L} \log(\hat{p}_{1d_L})$$

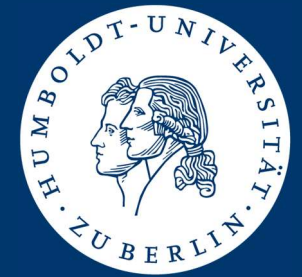
Binary Cross Entropy

- Special case for two-class problems
- Loss Function for binary classification
- Equivalent to log-loss
(aka negative log-likelihood)

$$J(W) = - \sum_{i=1}^n \sum_{c=1}^{d_L=2} y_{ic} \log(f_W(\mathbf{X}_i))$$

$$= - \sum_{i=1}^n y_{i1} \log(\hat{p}_{i1}) + (1 - y_{i1}) \log(1 - \hat{p}_{i1})$$





Neural Network Training in Practice

Gradient descent revisited, learning rates, and other tweaks

Neural Network Training in Practice

Gradient descent revisited

■ **Initialize weights randomly** $\sim N(0, \sigma^2)$

■ **Loop until convergence**

□ Compute forward path and loss $\hat{y} = f_{\mathbf{W}_t}(\mathbf{x})$ $J = (\mathbf{y} - \hat{y})^2$

□ Compute gradient, $\frac{\partial J(\mathbf{W}_t)}{\partial \mathbf{W}_t}$ Above, we assumed (implicitly) that we compute the loss based on one training example.

□ Update network weights, $\mathbf{W}_{t+1} \leftarrow \mathbf{W}_t - \eta \frac{\partial J(\mathbf{W}_t)}{\partial \mathbf{W}_t}$

■ **Return weights**

Neural Network Training in Practice

Different flavors of gradient descent

■ Initialize weights randomly $\sim N(0, \sigma^2)$

■ Loop until convergence

□ Compute forward path and loss $\hat{y}_i = f_W(\mathbf{x})$ $J = (y_i - \hat{y}_i)^2$

□ Compute gradient, $\frac{\partial J(W)}{\partial W}$ This specific form of gradient descent is called (pure) **stochastic** gradient descent (SGD).

□ Update network weights, $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$

■ Return weights

Neural Network Training in Practice

Different flavors of gradient descent

■ Initialize weights randomly $\sim N(0, \sigma^2)$

■ Loop until convergence

□ Compute forward path and loss $\hat{y}_i = f_W(\mathbf{x})$

$$J = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

□ Compute gradient,

$$\frac{\partial J(W)}{\partial W}$$

Alternatively, we could compute the loss on the entire training set. This is called **batch gradient descent**.

□ Update network weights,

$$W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$$

■ Return weights

Batch vs. Stochastic Gradient Descent

Pros and cons

■ Gradient identifies direction in which a function (i.e., the loss) increases the most

- Direction of steepest ascent
- Hence updating weights in the direction of the negative of the gradient

■ Batch gradient descent

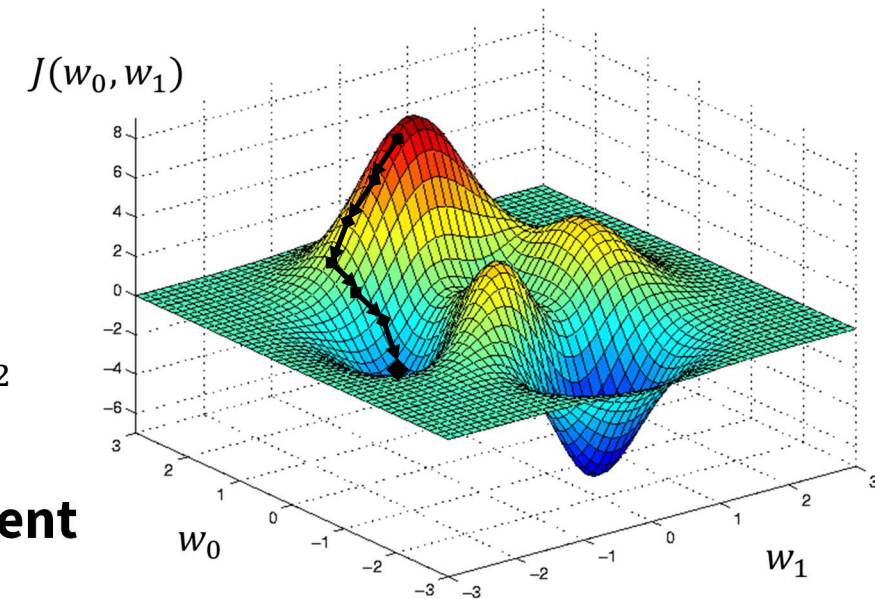
- Accurate approximation of the gradient
- Very costly to compute because requires one pass through entire training set

$$J = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

■ Opposite pros/cons with stochastic gradient descent

■ Some terminology

- A path to the training set is called an **epoch**
- Batch gradient descent updates weights once per epoch → NN learns slower, training will take longer
- SGD updates weights once per data point → Fast learning but risk of bad updates due to noisy gradient



Mini-Batch Gradient Descent

The compromise and prevailing approach in practice

■ Decide on a batch size B

- Meta-parameter of NN training
- May need some tuning

■ Loop until convergence

- Run B training examples through the NN

- Compute loss

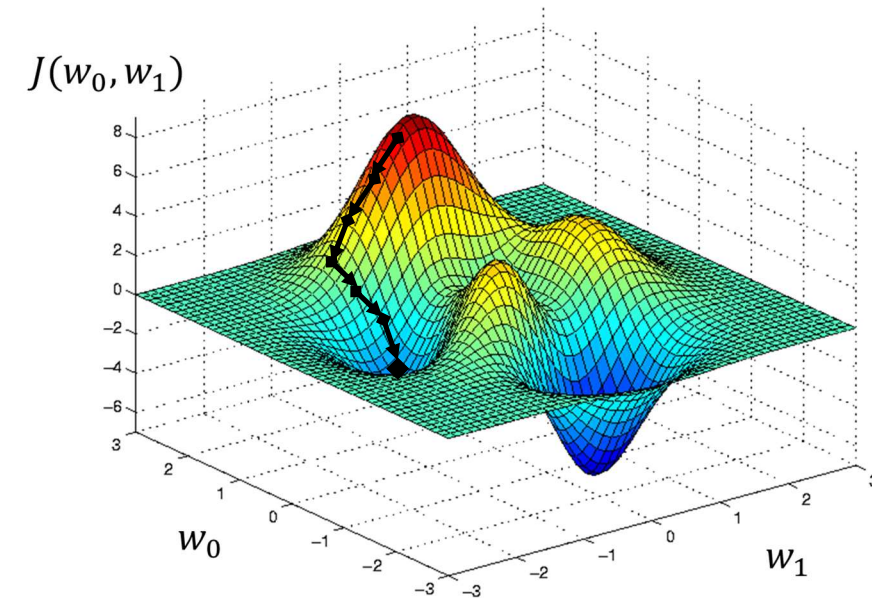
$$J = \sum_{i=1}^B (y_i - \hat{y}_i)^2$$

- Compute (i.e., approximate) gradient based on the loss

$$\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}} = \frac{1}{B} \sum_{k=1}^B \frac{\partial J_k(\mathbf{W})}{\partial \mathbf{W}}$$

- Update weights

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$$



Mini-Batch Gradient Descent

Discussion

■ Can be considered standard practice (for larger data sets)

■ Advantages

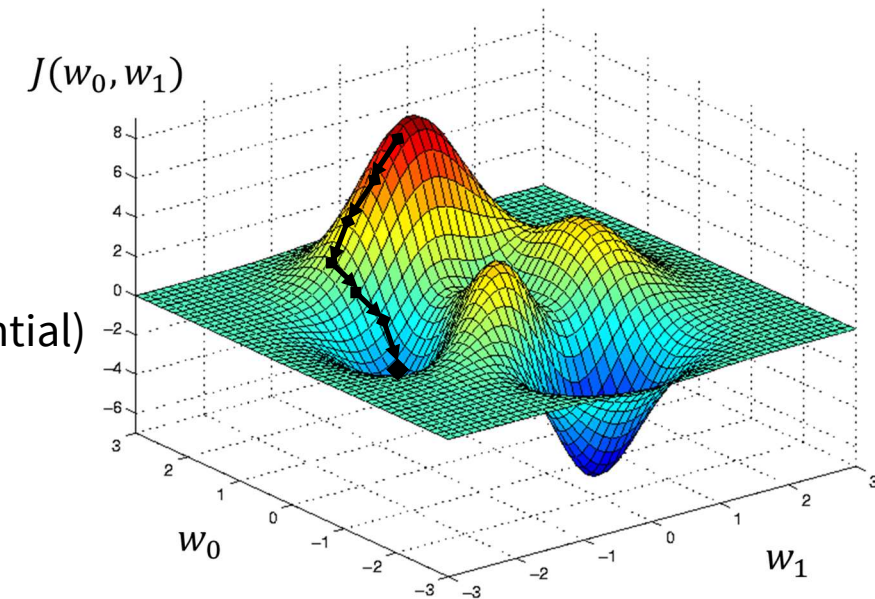
- More accurate estimation of the gradient than (pure) SGD
- Smoother convergence
- Allows for larger learning rates
- Allows for parallelization (speed-up on GPU's can be substantial)

■ Disadvantages

- Need to decide on mini-batch size
- May take longer than batch SGD for smaller data sets

■ Further reading

- S. Ruder, “An overview of gradient descent optimization algorithms”
- <http://ruder.io/optimizing-gradient-descent/>

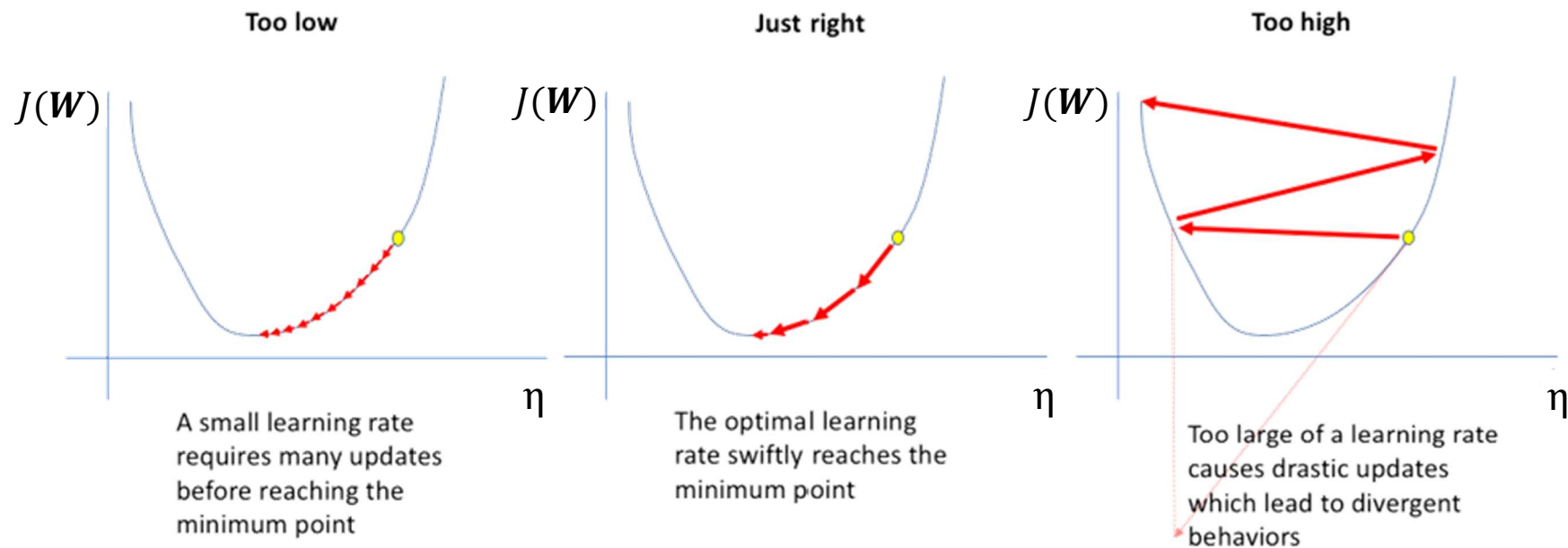


Speaking About Learning

Considerations on setting the learning rate

- Optimization through gradient descent
- Speed of convergence dependent on η

$$W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$$



Source: <https://www.jeremyjordan.me/nn-learning-rate/>

Adaptive Learning Rates

Learning rate not fixed but adjusts to error surface

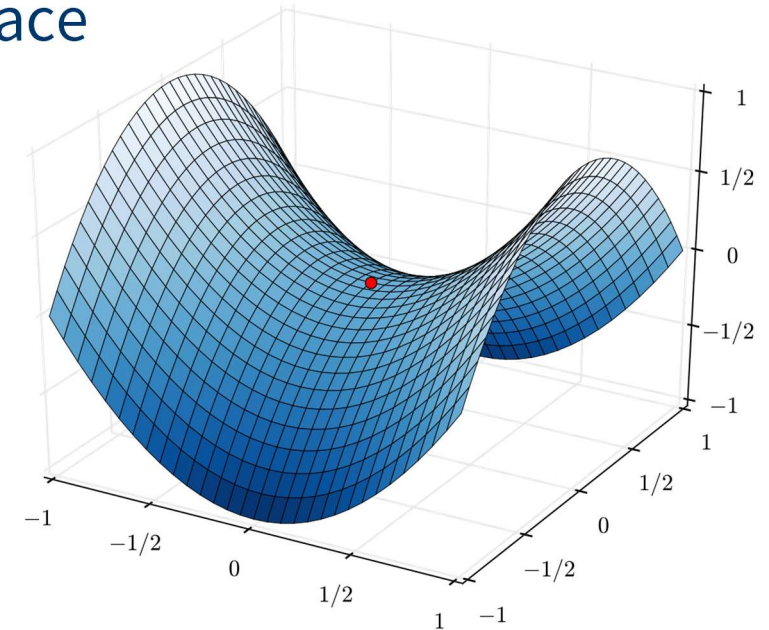
■ In-/decrease learning rate depending on:

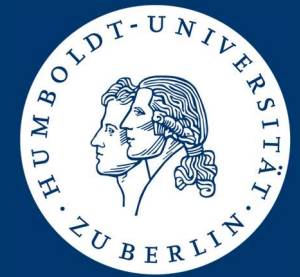
- Magnitude of the gradient
- Speed of learning
- Size of (particular) weights
- ...

■ Active field of research

■ Adaptive learning rate algorithms

- Momentum Qian et al. “On the momentum term in gradient descent learning algorithms.” 1999.
- Adagrad Duchi et al. “Adaptive Subgradient Methods for Online Learning and Stochastic Optimization.” 2011.
- Adadelta Zeiler et al. “ADADELTA: An Adaptive Learning Rate Method.” 2012.
- Adam Kingma et al. “Adam: A Method for Stochastic Optimization.” 2014.
- RMSProp Hinton et al. „Neural Networks for Machine Learning.“ 2014.





Overfitting in Neural Networks

Bias variance tradeoff, regularization, early stopping and dropout

Overfitting

One of the key issues in supervised machine learning

■ **Supervised models aim at accuracy (forecasts, classifications, detections, ...)**

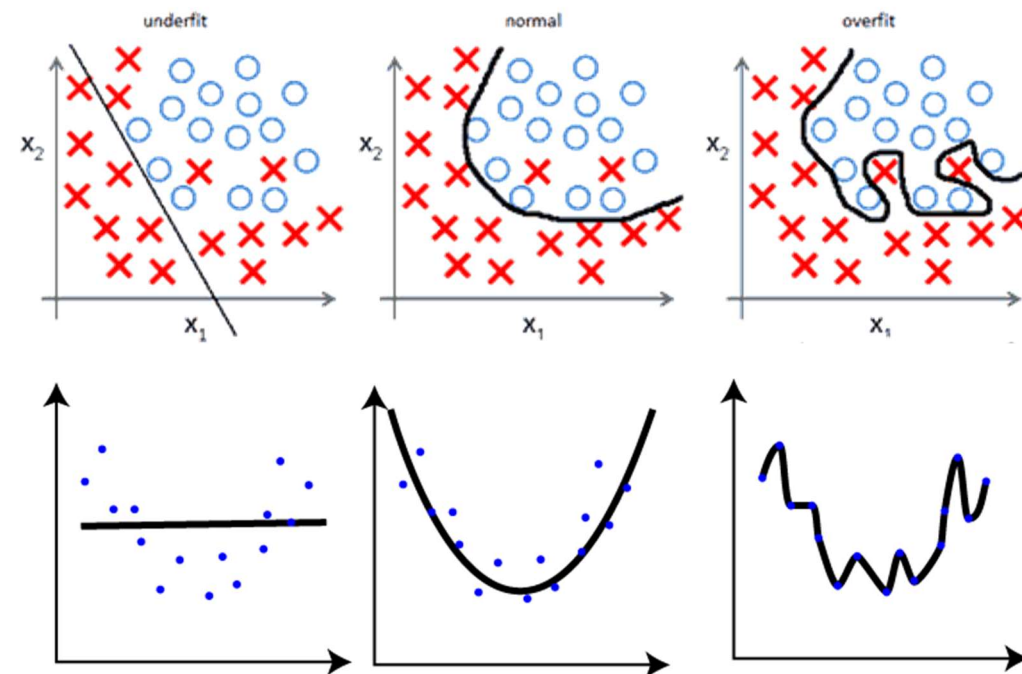
■ **Focus of interest – where accuracy matters – is deployment environment**

- Conversion model → all surfers visiting our website
- Credit approval model → all incoming applications
- Tweet sentiment classifier → all Twitter postings
- ...

■ **We build – train – models on a sample**

■ **Overfitting**

- Model fits training data very well
- Because of capturing both, structure and noise
- Trying to forecast the noise, which is random, the model produces poor results out-of-sample



Formal Perspective on Overfitting Through Bias-Variance

Decomposition of the Generalization Error

- Let $Y = f(X) + \epsilon$, with random variables

- $Y \in \mathbb{R}, X \in \mathbb{R}^m$
- $\epsilon \sim \mathcal{N}(0, \sigma^2) \in \mathbb{R}$

- Generalization error of ML model $\hat{f}_D(X)$

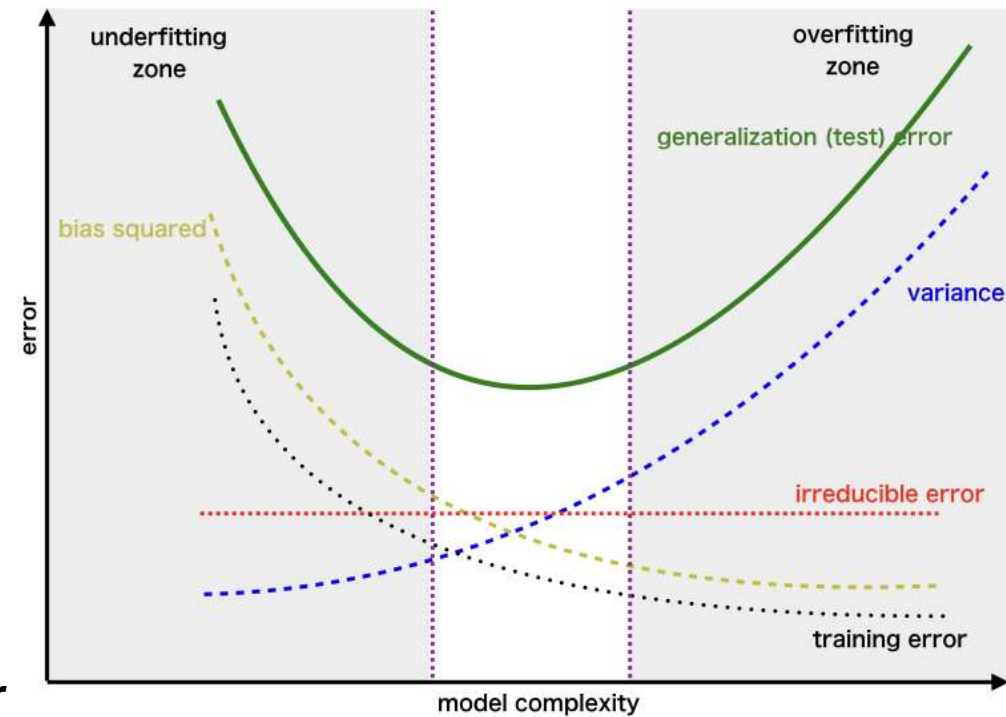
- $E_{D, \epsilon} \left[\left(Y - \hat{f}_D(X) \right)^2 \right]$
- Expectation over data sets $\mathcal{D} = \{(X_1, Y_1), \dots, (X_n, Y_n)\}$ sampled from unknown joint distribution $P(X, Y)$

- Bias and Variance of the ML model

- $\text{Bias}_D[\hat{f}_D(X)] = E_D[\hat{f}_D(X) - f(X)]$
- $\text{Var}_D[\hat{f}_D(X)] = E_D \left[\left(\hat{f}_D(X) - E_D[\hat{f}_D(X)] \right)^2 \right]$

- Bias-variance decomposition of generalization error

- $E_{D, \epsilon} \left[\left(Y - \hat{f}_D(X) \right)^2 \right] = (\text{Bias}_D[\hat{f}_D(X)])^2 + \text{Var}_D[\hat{f}_D(X)] + \sigma^2$
- Overfitting occurs if a model has low bias and high variance



Regularization

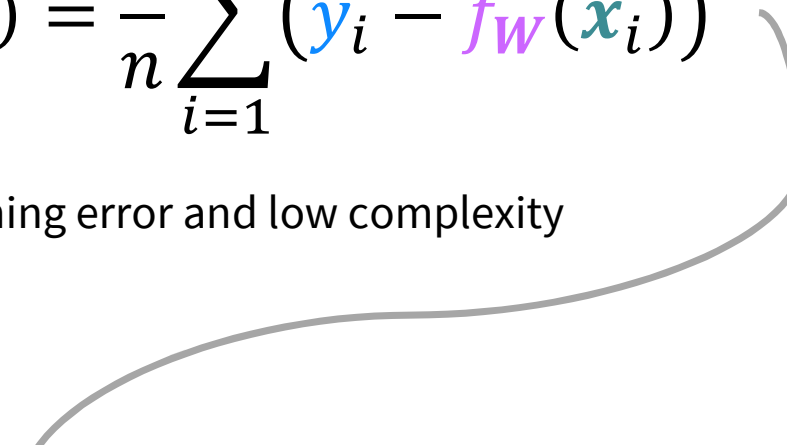
Introduce bias to reduce variance (and find a better balance between them)

- Add a term to the loss function that penalizes complexity
- Much of the complexities in NNs stems from the layered architecture with many free parameters (i.e., connection weights)

- To regularize a NN, penalize the weights

$$J(\mathbf{W}) = \frac{1}{n} \sum_{i=1}^n (y_i - f_{\mathbf{W}}(x_i))^2$$

- Also called weight decay in NN learning
- Approximate 'complexity' as the total sum over weights
- Add meta-parameter to control the trade-off between low training error and low complexity

$$J'(\mathbf{W}) = \frac{1}{n} \sum_{i=1}^n J(\mathbf{W}) + \lambda \|\mathbf{W}\|$$


Regularization

Common penalty functions

■ Ridge aka L2 penalty

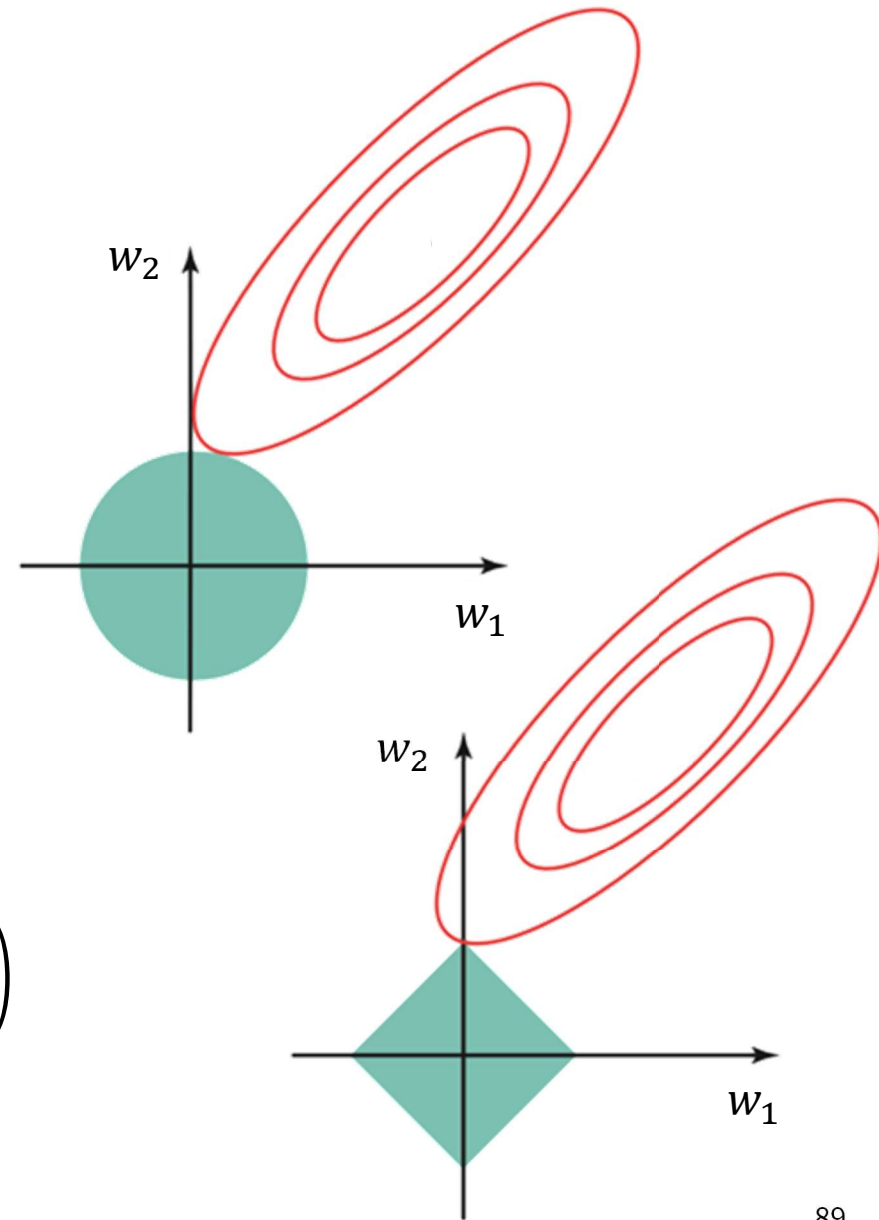
$$\|W\|_2 = \lambda \sum_j w_j^2$$

■ LASSO aka L1 penalty

$$\|W\|_1 = \lambda \sum_j |w_j|$$

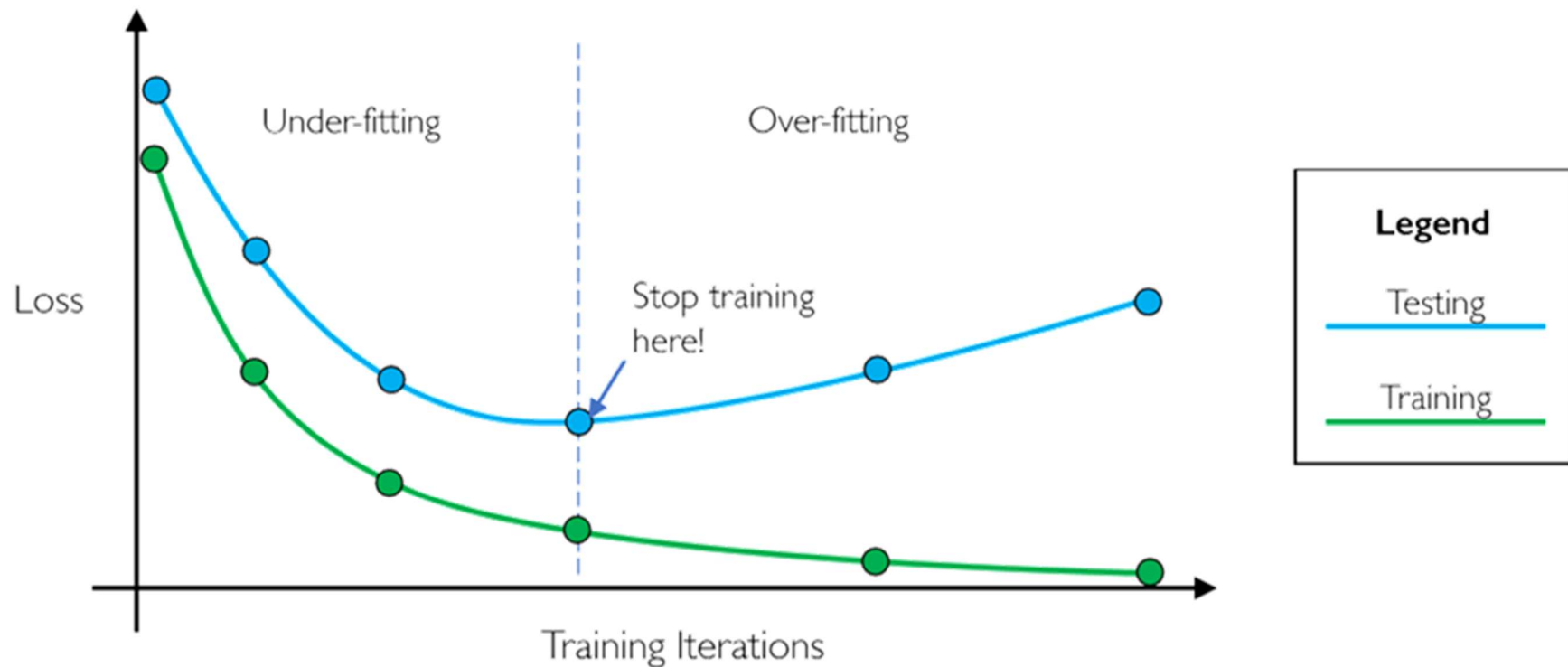
■ Elastic net penalty

$$\|W\|_{enet} = \lambda \left(\frac{1-\alpha}{2} \sum_j w_j^2 + \alpha \sum_j |w_j| \right)$$



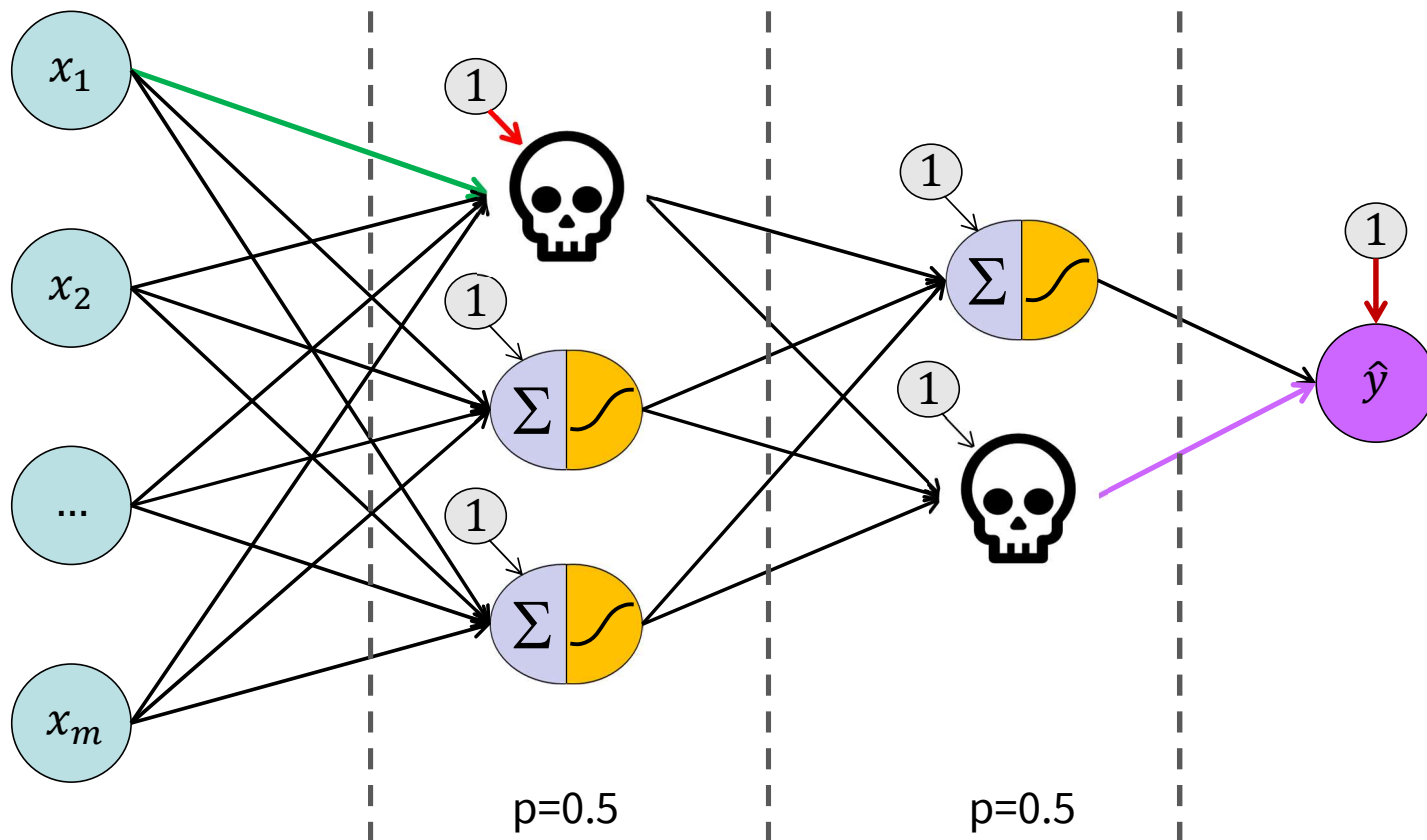
Early Stopping

- Holdback a subset of the training data for validation
- Trace loss on validation sample during network training
- Terminate training before network weights adjust too much to training data



Dropout: A Simple Way to Prevent Neural Networks from Overfitting

Stochastically deactivate some hidden units during training

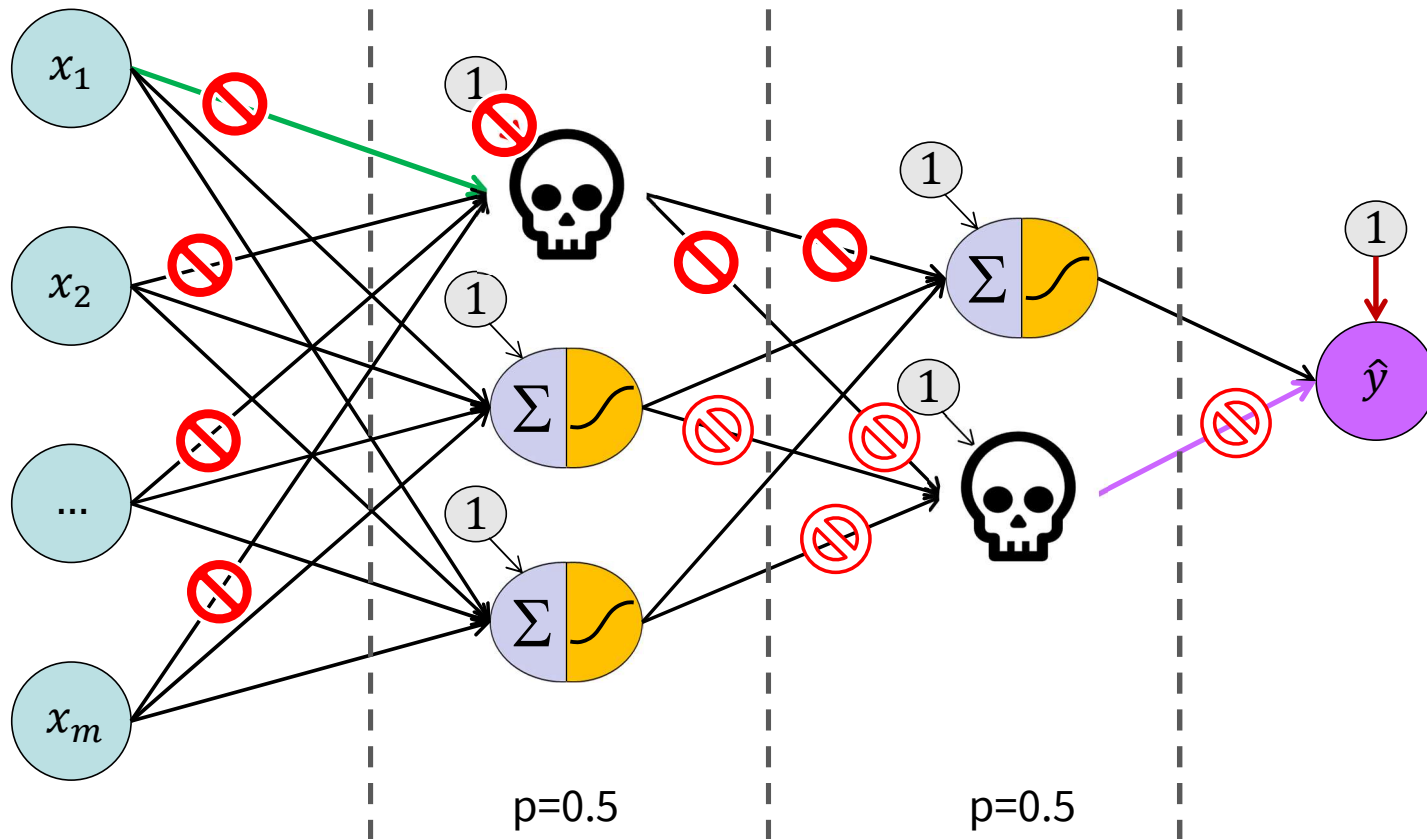


Dropout rate:

probability to deactivate a node in a given hidden layer of the NN

Dropout: A Simple Way to Prevent Neural Networks from Overfitting

A dropped-out node blocks the flow of information over certain connections

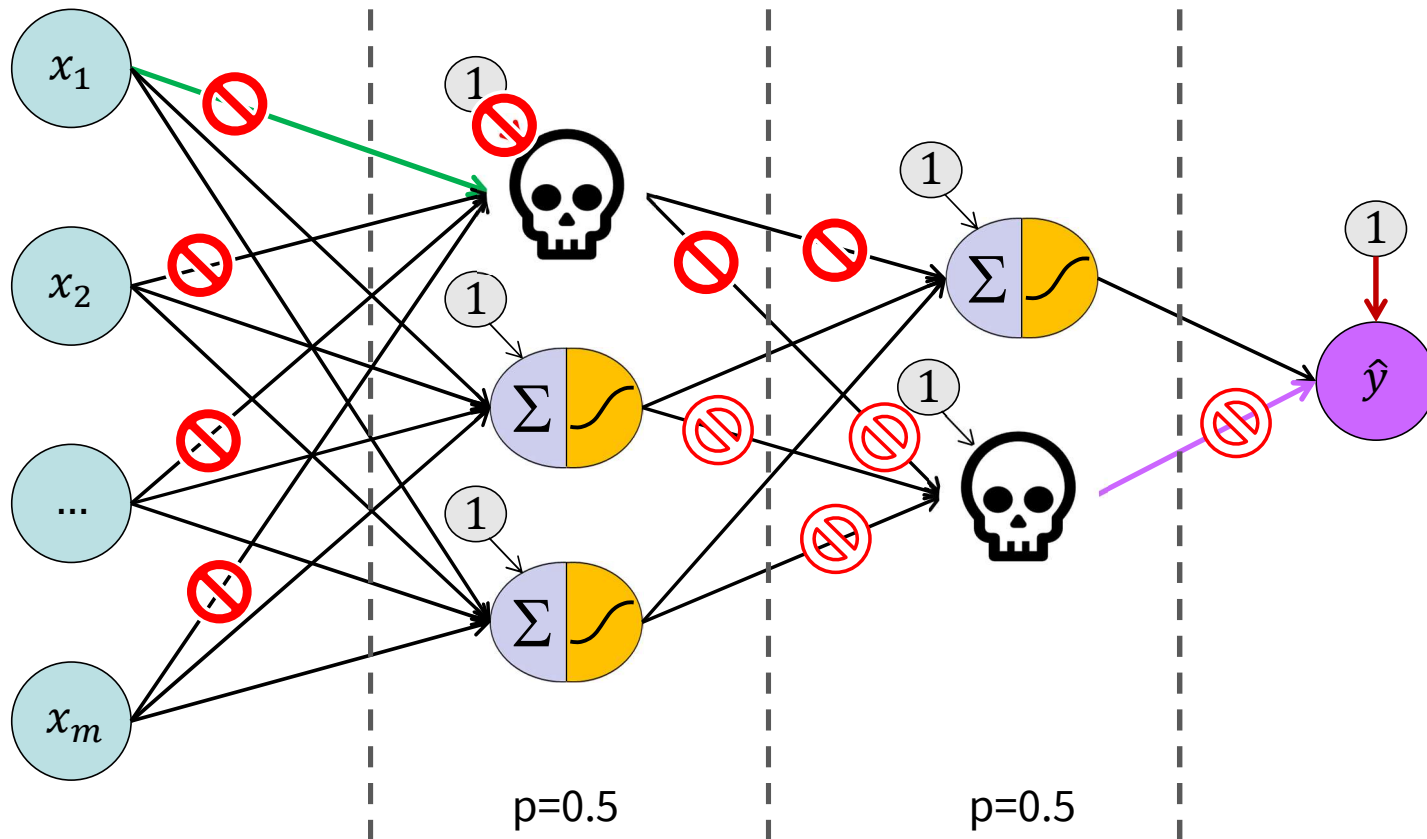


Dropout rate:

probability to deactivate a node in a given hidden layer of the NN

Dropout: A Simple Way to Prevent Neural Networks from Overfitting

The weights of those connections will not get trained



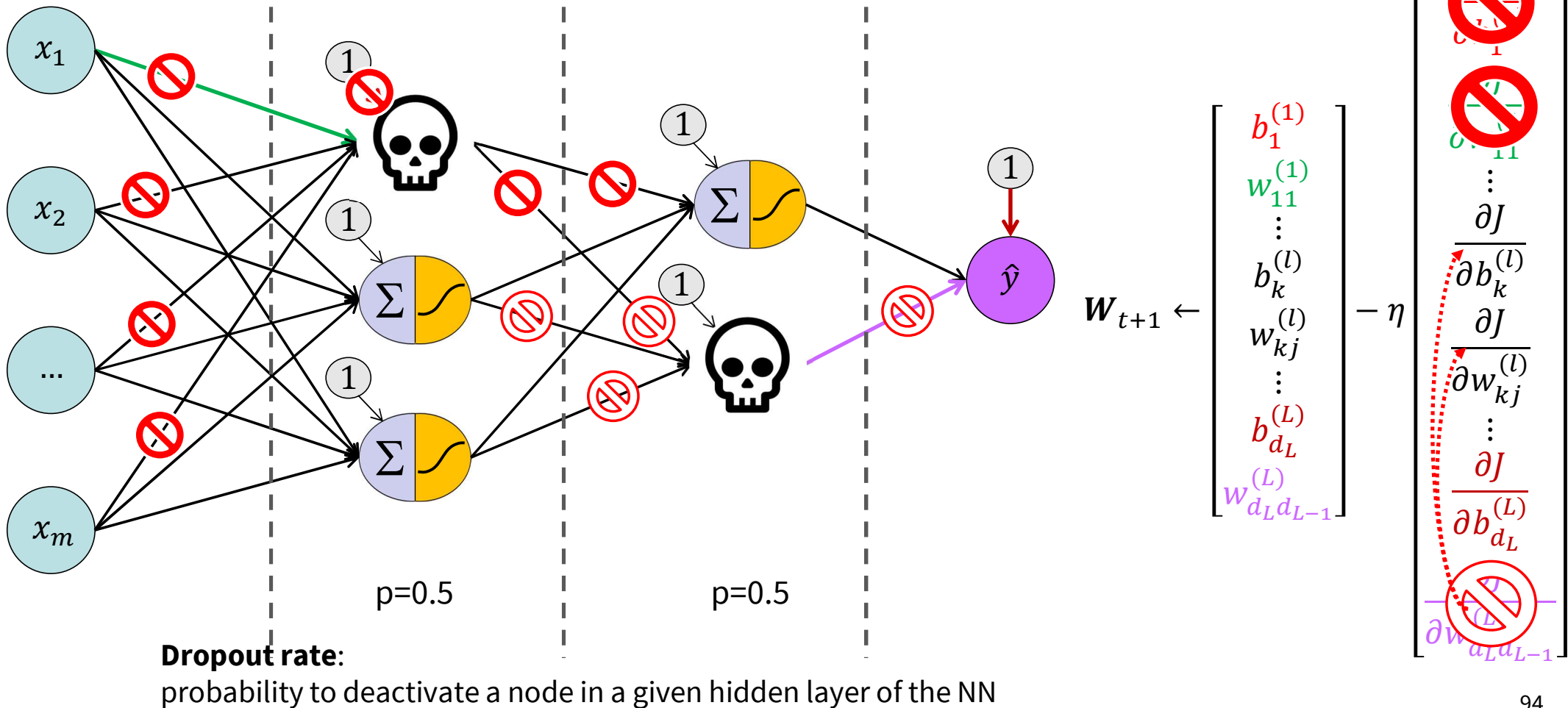
Dropout rate:

probability to deactivate a node in a given hidden layer of the NN

$$\frac{\partial J(W)}{\partial W} = \begin{bmatrix} \frac{\partial J}{\partial b_1^{(l)}} \\ \frac{\partial J}{\partial b_k^{(l)}} \\ \vdots \\ \frac{\partial J}{\partial w_{kj}^{(l)}} \\ \vdots \\ \frac{\partial J}{\partial b_{d_L}^{(L)}} \\ \frac{\partial J}{\partial w_{d_L d_{L-1}}} \end{bmatrix}$$

Dropout: A Simple Way to Prevent Neural Networks from Overfitting

This also effects the training over other weights due to backpropagation



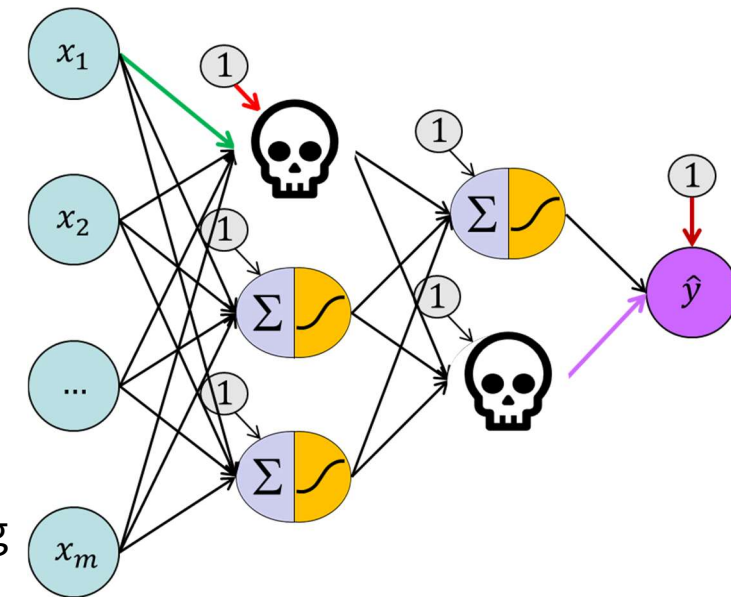
Dropout in a Nutshell

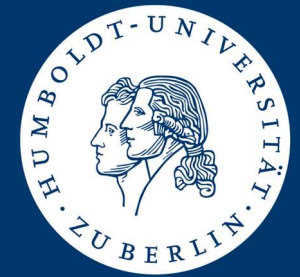
■ Each hidden unit is set to zero with probability p , often set to 50 percent

- Applied for each batch of examples or weight update
- Gradient of that batch by-passes dropped-out neurons
 - Hidden nodes cannot co-adapt to other nodes
 - Hidden nodes must be more generally useful
- Forces network to not rely on any one node

■ Effectively, dropout gives an ensemble

- Training is done across many NNs with different architecture
- If a node is retained with probability p during training, the outgoing weights of that node are multiplied by p at test time
- Like automatic model averaging with beneficial effect on variance





Further Tweaks and Tuning

Data preparation, weight initialization and model selection

Data Preparation for Neural Networks

Everything covered in BADS also applies to neural networks

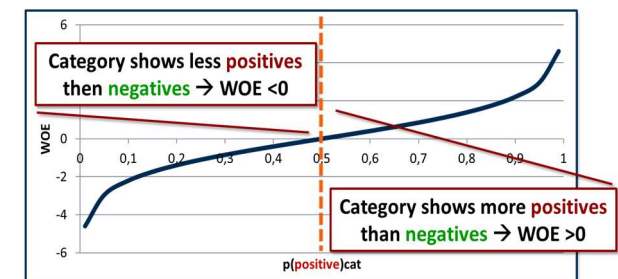
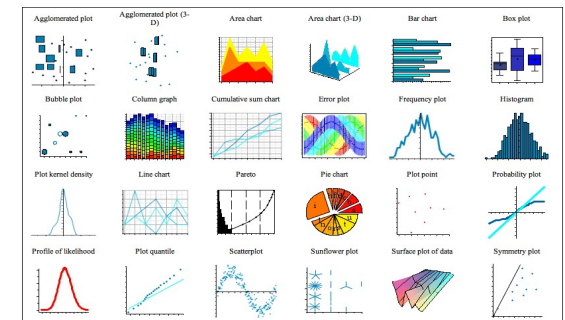
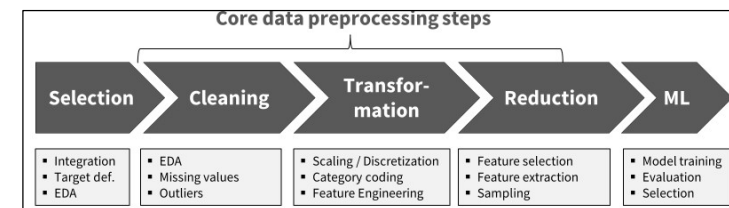
- **Criticality of EDA for data understanding**
- **Variable selection to reduce dimensionality**
- **Treatment of categories**

- Neural networks require numeric inputs
- Dummy coding
- Weight-of-evidence for binary classification
- Regrouping using, e.g., information gain

- **See BADS course for details**

- **Feature engineering debatable**

- See later for automatic feature learning in deep NNs
- Purposefully crafted features still very useful
- Might depend on problem setting (e.g., structured vs. unstructured data)



Data Preparation for Neural Networks

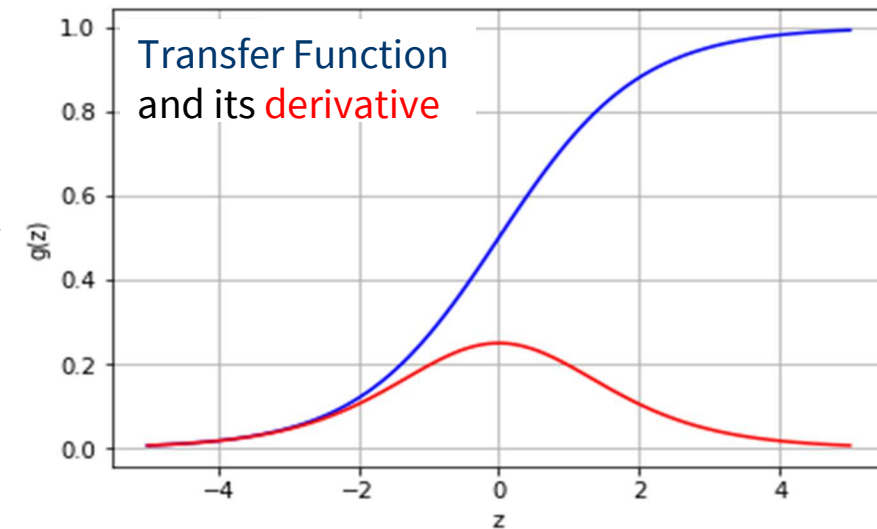
Scaling of continuous input variables is crucial

■ Values of all features in a similar number range

- Age, salary, loan amount, ...
- Min/Max Scaling or Z-Transformation

■ NN training much slower w/o scaling

- Problem of saturated neurons
- Neuron input calculated as feature value * weight
 - Large (unscaled) feature value like List Price=1,250
 - Times a random initial weight $w \in (0,1)$
- Neuron output likely close to 0 or 1
- Partial derivative in that range is essentially zero
- Partial derivative governs NN training
 - Weight updates in gradient descent
 - Partial derivative of a weight virtually zero
 - Virtually no update of that weight



	Input $l = 1$	Hidden $1 < l < L$	Output $l = L$
Calculation of δ	$\delta^{(l)} \equiv \delta^{(l+1)} w^{(l+1)} g'(z^{(l)})$		$\delta^{(L)} = \frac{\partial J}{\partial \hat{y}} \times \dots$
Partial derivative of w	$\frac{\partial J}{\partial w^{(l)}} = \delta^{(l)} \times h^{(l-1)}$		
Partial derivative of b	$\frac{\partial J}{\partial b^{(l)}} = \delta^{(l)}$		

$$\mathbf{W}_{t+1} \leftarrow \mathbf{W}_t - \eta \frac{\partial J(\mathbf{W}_t)}{\partial \mathbf{W}_t} \Leftrightarrow \mathbf{W}_{t+1} \leftarrow \begin{bmatrix} w_{11}^{(1)} \\ \vdots \\ w_{kj}^{(l)} \\ \vdots \\ w_{d_L d_{L-1}}^{(L)} \end{bmatrix} - \eta \begin{bmatrix} \partial J / \partial w_{11}^{(1)} \\ \vdots \\ \partial J / \partial w_{kj}^{(l)} \\ \vdots \\ \partial J / \partial w_{d_L d_{L-1}}^{(L)} \end{bmatrix}$$

Data Preparation for Neural Networks

Input normalization summary

■ Theoretically not strictly necessary

- Network could adjust weights and bias
- Gives the same input as before scaling

■ Practically useful

- Speed-up training
- Avoid neuron saturation
- Avoid getting stuck in local minima

■ Scaling is mandatory if using weight decay

■ Depends on activation function

■ No ultimate rule

- Standardization should work in many cases
- Avoid input values in the saturation level
 - $g(z) \in [0,1]$ for logit
 - $g(z) \in [-1,1]$ for tanH
- Use min/max scaling and project data to, e.g.,
 - $[0.1; 0.9]$ for logit
 - $[-0.7; +0.7]$ for tanH

■ RELU should be more robust

- Positive gradient for $z > 0$

Weight Initialization

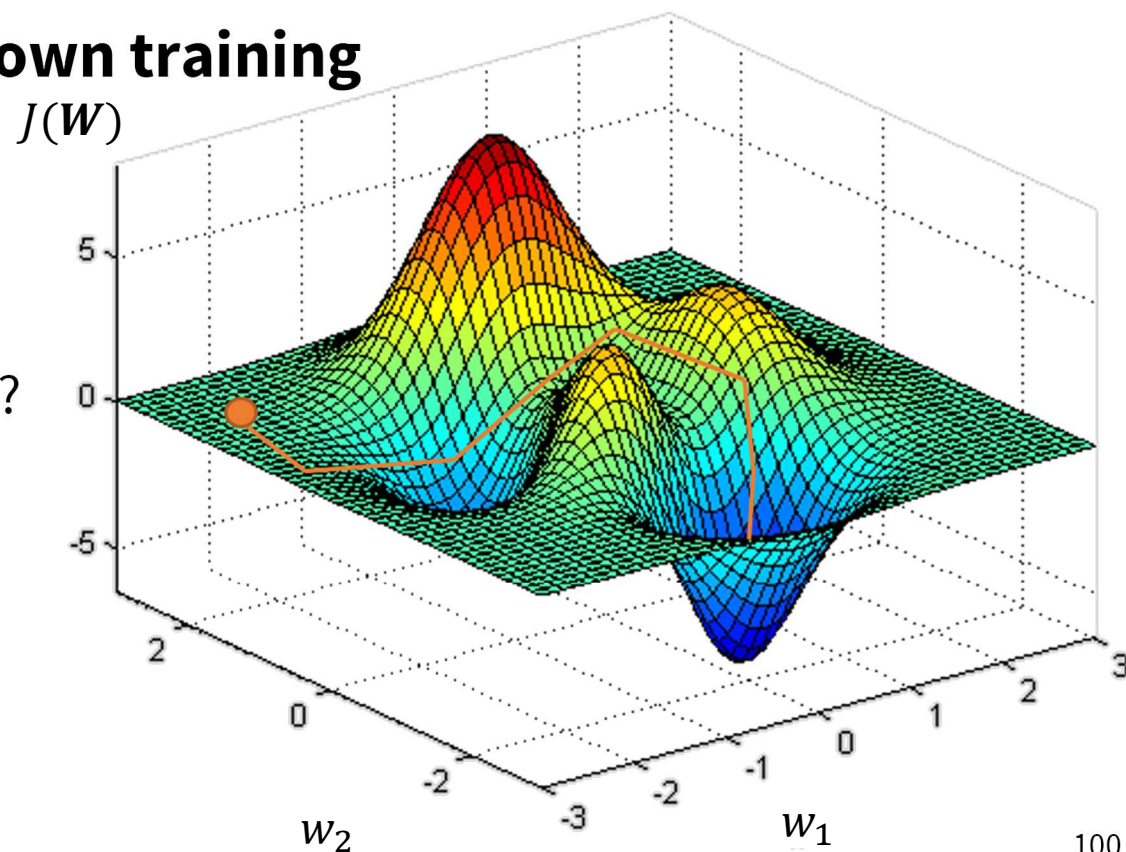
- “Good” initial weights can accelerate network training

- “Bad” initial weights can slow down training (or prohibit it altogether)

- But...

- How to find “good” weights?

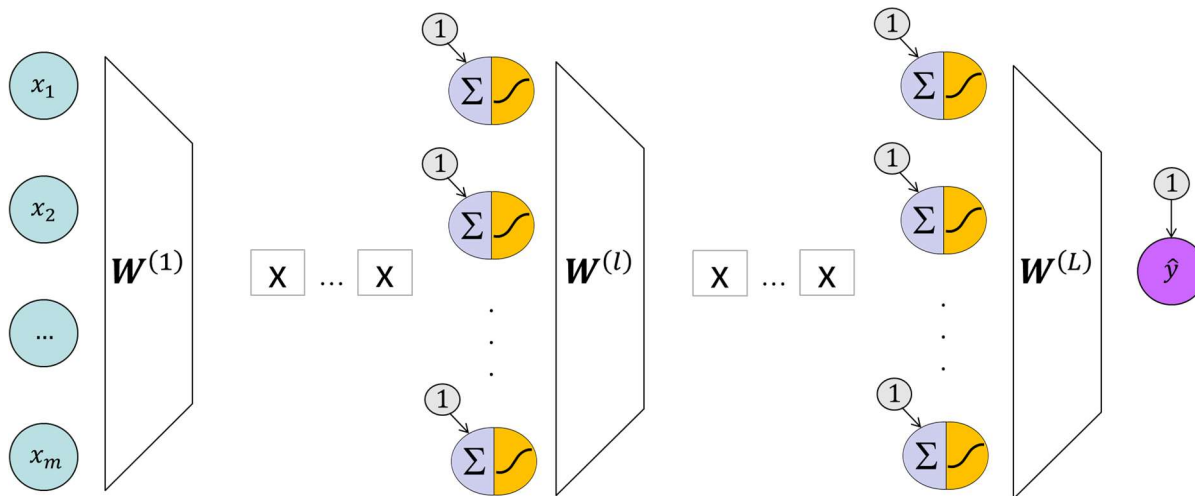
- And what actually are “good weights”?



Weight Initialization

Desirable properties of initial network weights

- Forward pass corresponds to a chain of calculations
- Iterating sequence of multiplying matrixes and adding nonlinearity



$$Z^{(l)} = \begin{bmatrix} z_1^{(l)} \\ \vdots \\ z_{d_l}^{(l)} \end{bmatrix} = B^{(l)} + W^{(l)} H^{(l-1)}$$

$$H^{(l)} = \begin{bmatrix} h_1^{(l)} \\ \vdots \\ h_{d_l}^{(l)} \end{bmatrix} = g(Z^{(l)}) = \begin{bmatrix} g(z_1^{(l)}) \\ \vdots \\ g(z_{d_l}^{(l)}) \end{bmatrix}$$

Weight Initialization

Desirable properties of initial network weights

- **Forward pass corresponds to a chain of calculations**
- **Iterating sequence of multiplying matrixes and adding nonlinearity**
- **Simulation experiment**
 - Network of 100 layers with 512 units each
 - No non-linear activation functions (for simplicity)
 - Standardized input vector (drawn at random, $x \sim N(0,1)$)
 - Random weight matrix $W \sim N(0,1)$
 - Compute forward pass and check result

```
import matplotlib.pyplot as plt
import numpy as np

# Generate random input data using the normal distribution
m = 512
x = np.random.normal(size=m)
x = x.reshape((m,1))

# Compute forward pass through 100 layers without non-linear activation
# (just matrix multiplication)
for i in range(100):
    W = np.random.normal(size=(m,m))
    x = np.dot(W,x)
# check moments of the resulting output
print("Mean of x is {:.2e} with std. deviation of {:.2e}".format(x.mean(), x.std()))
```

Mean of x is 8.22e+133 with std. deviation of 2.68e+135

Weight Initialization

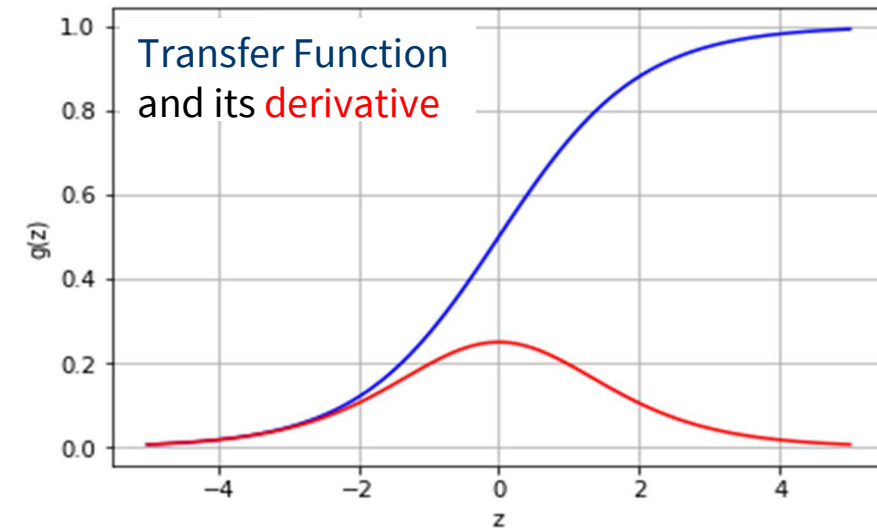
Desirable properties of initial network weights

■ Simulation experiment summarized

- Initial weights too large
 - Activation signal grows with each layer
 - Approaches infinity
- Initial weights too small
 - Activation signal shrinks with each layer
 - Approaches zero

■ Implications for training

- Gradient explosion → large weight updates
- Gradient vanishing → no weight updates
- Both cases impede network training



	Input $l = 1$	Hidden $1 < l < L$	Output $l = L$
Calculation of δ	$\delta^{(l)} \equiv \delta^{(l+1)} w^{(l+1)} g'(z^{(l)})$		$\delta^{(L)} = \frac{\partial J}{\partial \hat{y}} \times \dots$
Partial derivative of w	$\frac{\partial J}{\partial w^{(l)}} = \delta^{(l)} \times h^{(l-1)}$		
Partial derivative of b	$\frac{\partial J}{\partial b^{(l)}} = \delta^{(l)}$		

$$\mathbf{W}_{t+1} \leftarrow \mathbf{W}_t - \eta \frac{\partial J(\mathbf{W}_t)}{\partial \mathbf{W}_t} \Leftrightarrow \mathbf{W}_{t+1} \leftarrow \begin{bmatrix} w_{11}^{(1)} \\ \vdots \\ w_{kj}^{(l)} \\ \vdots \\ w_{d_L d_{L-1}}^{(L)} \end{bmatrix} - \eta \begin{bmatrix} \partial J / \partial w_{11}^{(1)} \\ \vdots \\ \partial J / \partial w_{kj}^{(l)} \\ \vdots \\ \partial J / \partial w_{d_L d_{L-1}}^{(L)} \end{bmatrix}$$

Weight Initialization

Xavier Initialization

■ Explosion/vanishing of activation levels rooted in variance of network weights

- Imagine hidden layer with k neurons initialized using standard normal
- Next layer input calculated as weighted sum over the k activation values
- Variance in the next layer approximately equal to k

```
m=512
n=100
mean, var = 0., 0.
for i in range(n):
    x = np.random.normal(size=(m,1))
    W = np.random.normal(size=(m,m))
    y = np.dot(W,x)
    mean += y.mean().item()
    var += y.var().item()
```

■ Solve problem through controlling variance of network weights

- Approximately constant variance when passing layers in forward computation
- $var(out) = var(in) = var(w_1x_1 + w_2x_2 + \dots + w_mx_m + b)$
- One can show that to meet this requirement

$$var(W) = 1/N_{in}$$

$$var(W) = \frac{2}{N_{in} + N_{out}}$$

■ More specifically

- Gaussian or uniform distribution with zero mean and variance
- Original proposition from paper (accounts for forward and backward pass)

X. Glorot & J. Bengio (2010), "Understanding the difficulty of training deep feedforward neural networks", in: Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS) 2010, Sardinia, Italy.

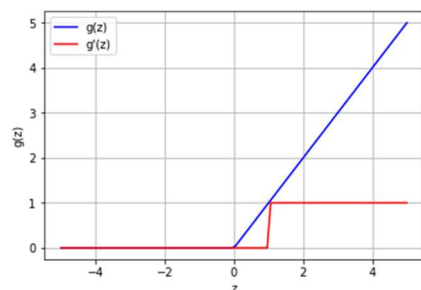
Weight Initialization

Kaiming Initialization

■ Xavier initialization suited for sigmoid, tanH, ... activations

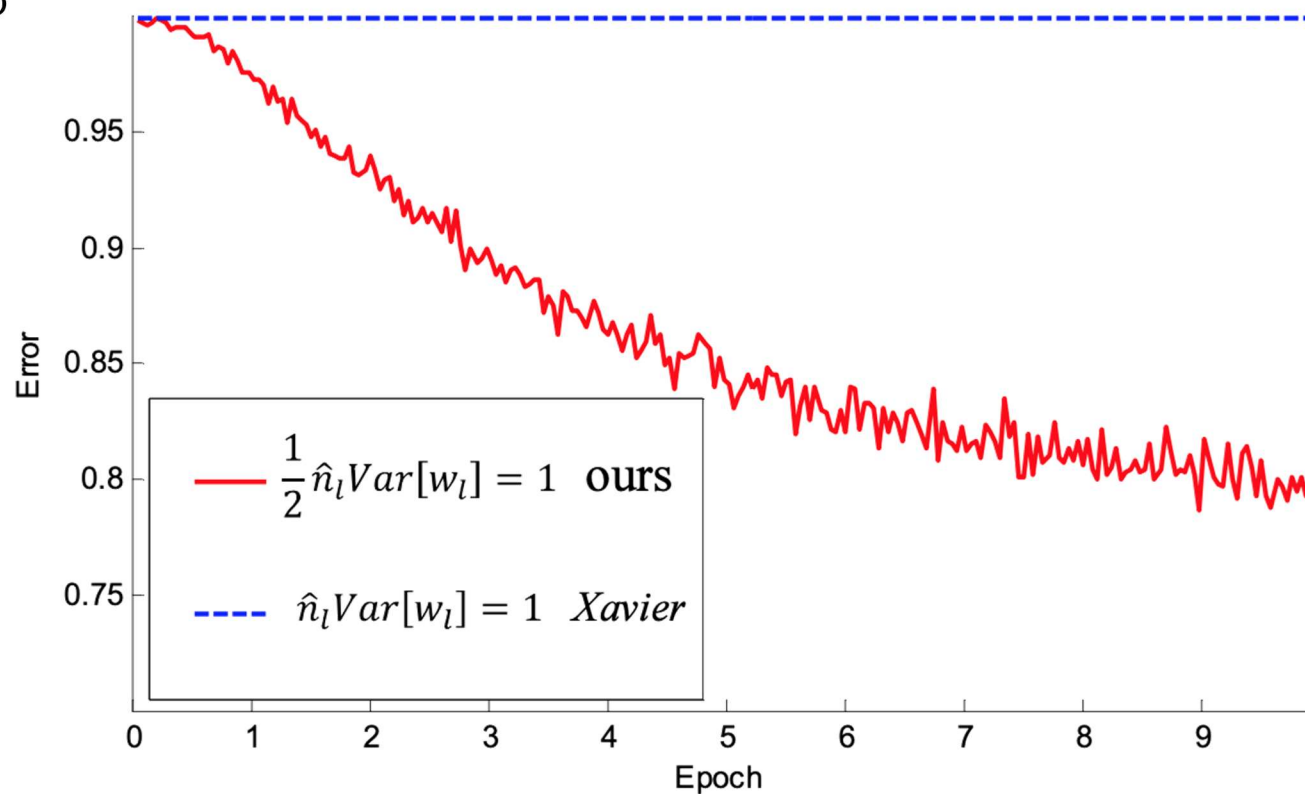
- Activation symmetric around zero
- Sensible to initialize weights around $[-1, +1]$

■ Less suitable for RELU



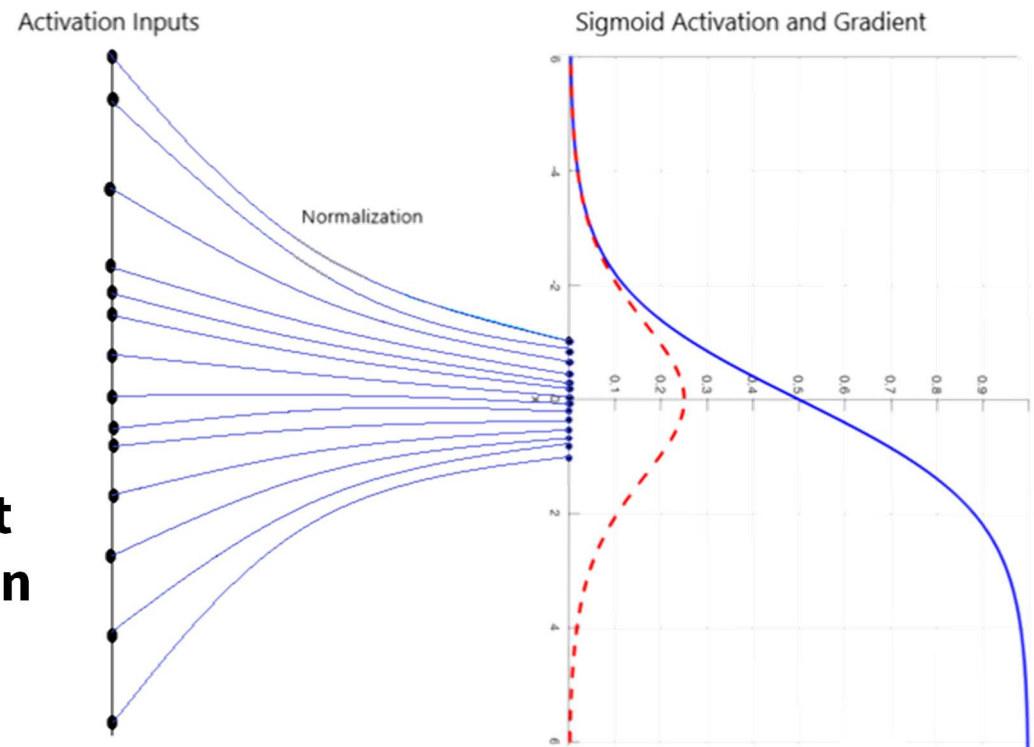
$$g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$



Batch Normalization

- Initialize weights in a “smart” way to avoid saturation
- Standardize input to avoid saturation
- But no direct control over „inputs“ of deeper layers
 - Deeper layers craft latent features
 - These could still grow too large (or small)
- Batch normalization layer to ensure that layer output falls in non-saturated region of the next layer’s activation function



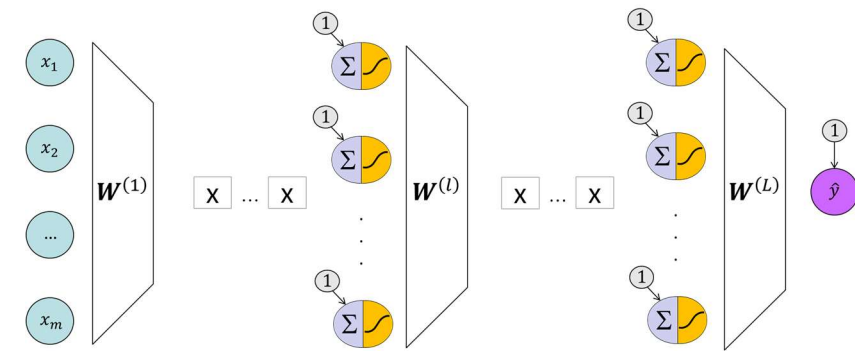
Neural Network Model Selection

Tune hyperparameters to optimize performance in a given setting

■ Neural networks belong to the family of semi-parametric models

■ Hyperparameters to be chosen by data analyst

- ☐ Number of hidden layers
- ☐ Number of units per hidden layer
- ☐ Hidden layer activation function
- ☐ Regularization
 - Which penalty L1, L2, Enet?
 - Early stopping?
 - Dropout? Dropout rate?
 - Layer-wise decisions possible
- ☐ Weight initialization
- ☐ Batch normalization
- ☐ Network training
 - Training algorithm
 - Number of epochs
 - Learning rate
 - Learning rate schedule
 - Momentum



$$Z^{(l)} = \begin{bmatrix} z_1^{(l)} \\ \vdots \\ z_{d_l}^{(l)} \end{bmatrix} = B^{(l)} + W^{(l)} H^{(l-1)}$$

$$H^{(l)} = \begin{bmatrix} h_1^{(l)} \\ \vdots \\ h_{d_l}^{(l)} \end{bmatrix} = g(Z^{(l)}) = \begin{bmatrix} g(z_1^{(l)}) \\ \vdots \\ g(z_{d_l}^{(l)}) \end{bmatrix}$$

Neural Network Model Selection

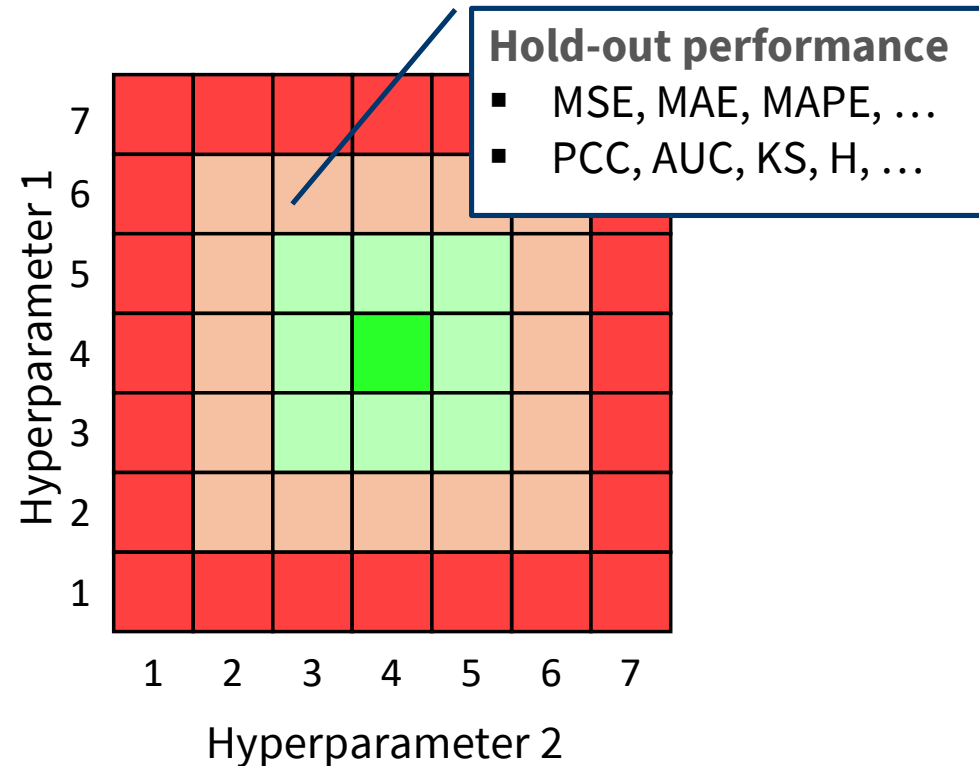
Grid Search

- **Fully enumerative search through all possible combinations of candidate hyperparameter settings**

- **Algorithm**

- ☐ Define candidate range for each hyperparameter
- ☐ Enumerate combinations of candidate values
- ☐ Train model with given configuration
- ☐ Assess model performance on hold-out data
- ☐ Repeat with next configuration

- **Magnify grid resolution in promising regions of the search space**



Neural Network Model Selection

Random Search

■ Grid Search is prohibitively expensive for (deep) neural networks

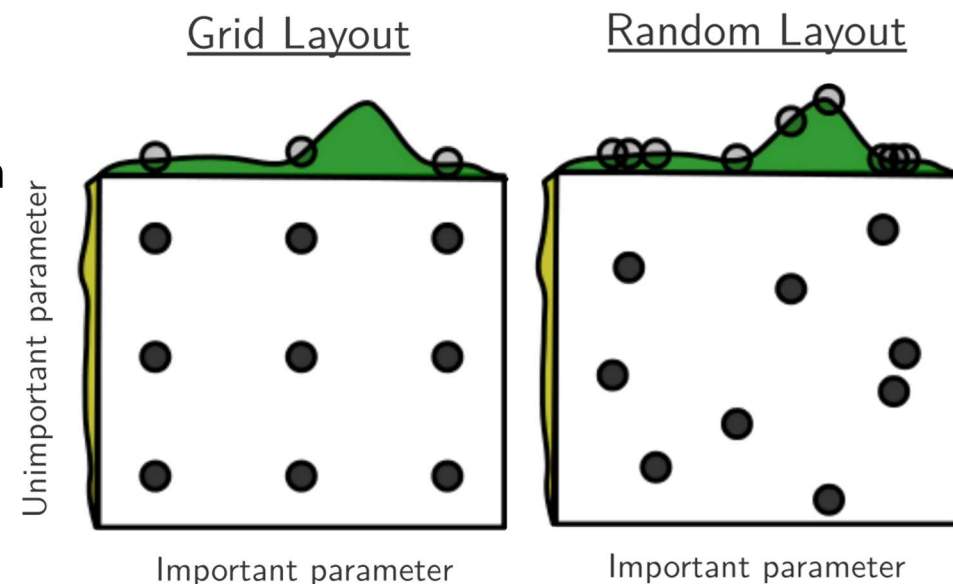
- To many distinct hyperparameters
- To many candidate values per hyperparameter

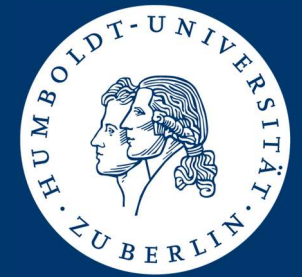
■ Better use **random search**

- Sample hyperparameter from a given prior distribution
- Bergstra & Bengio (2012) explore ways to set prior distributions. See also summary in Bengio (2012)

■ Algorithm

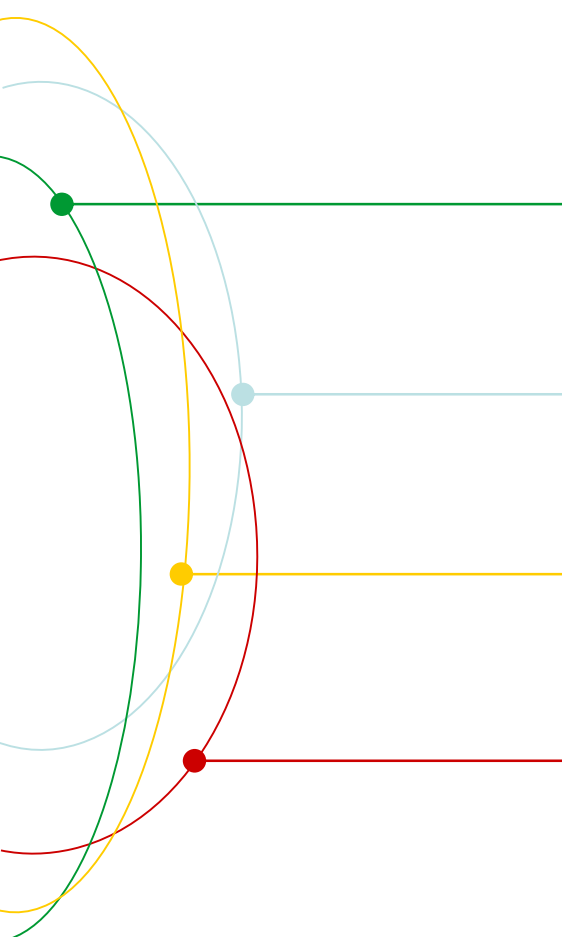
- Start from random initial solution
- Sample new solution
- Compare new to old and keep the better solution
- Repeat





Summary

Summary



Learning goals

- Overview of configuration options when setting up a neural network
- A bit of guidance how to deal with them



Findings

- Softmax & cross-entropy for classification
- Train NN using batch gradient descent
- Address overfitting regularization, dropout, and early stopping
- Scaling inputs & weights accelerates training
- More meta-parameters -> manual tweaking & random search



Homework

- Check out demo notebooks
- NN foundations & NN in keras



What next

- Start text analytics part
- Foundations of NLP

Credits & Disclaimer

- **The materials provided in the course have been inspired and draw upon numerous great sources available online**
- **The authors acknowledge the use of and wish to express their gratitude to the following sources (in alphabetical order)**
 - Machine Learning Glossary, <https://ml-cheatsheet.readthedocs.io/en/latest/index.html>
 - MIT 6.S191: Introduction to Deep Learning (<http://introtodeeplearning.com/>)
 - Medium All the Backpropagation derivatives (<https://medium.com/@pdquant/all-the-backpropagation-derivatives-d5275f727f60>)
 - Stanford CS224n: Natural Language Processing with Deep Learning (<https://web.stanford.edu/class/cs224n/>)
 - A. Zhang et al. : Dive into Deep Learning (<http://d2l.ai/index.html>)

Literature



- Bengio, Y. (2012), “Practical recommendations for gradient-based training of deep architectures”, in: Neural Networks: Tricks of the Trade. Springer, pp. 437–478.
- Bergstra, J., & Bengio, Y. (2012). Random Search for Hyper-Parameter Optimization Journal of Machine Learning Research, 13, 281-305.
- K. He et al. (2015), “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification, Archive Pre-Print 1502.01852

Thank you for your attention!

Stefan Lessmann

Chair of Information Systems
School of Business and Economics
Humboldt-University of Berlin, Germany

Tel. +49.30.2093.5742

Fax. +49.30.2093.5741

stefan.lessmann@hu-berlin.de

<http://bit.ly/hu-wi>

www.hu-berlin.de

